ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



HỆ ĐIỀU HÀNH LINUX VÀ ỨNG DỤNG

Bài thực hành

TRIỂN KHAI WEB SERVER APACHE TRÊN LINUX

Giảng viên hướng dẫn: Lê Hà Minh
Sinh viên thực hiện: Nguyễn Thành Dự - 24120288

Thành phố Hồ Chí Minh, 10/2026

Mục lục

1 Môi trường thực hiện **1**

1.1 Thông tin hệ thống 1

1.2 Sơ đồ hệ thống 1

2 Các phần thực hiện **2**

2.1 Phần 0 – Chuẩn bị môi trường 2

2.2 Phần A – Cài đặt và baseline 2

2.3 Phần B – Website đầu tiên trên Virtual Host (demo) 3

2.4 Phần C – Virtual Host thứ hai (demo2) 4

2.5 Phần D – Node API backend và Reverse Proxy (/api) 5

2.6 Phần E – Load Balancer (pool 3 backend + failover) 6

2.7 Phần F – SSL/TLS (HTTPS) + redirect + HSTS 7

2.8 Phần G – Tường lửa và AppArmor: rà soát và làm cứng 9

3 Bảng kiểm tra tổng **11**

4 Sự cố gặp phải và cách xử lý **12**

4.1 Không truy cập được HTTPS do Security Group thiếu cổng 443 12

4.2 Các điểm phát hiện thêm khi rà soát cấu hình 13

5 Kết luận **14**

1 Môi trường thực hiện

1.1 Thông tin hệ thống

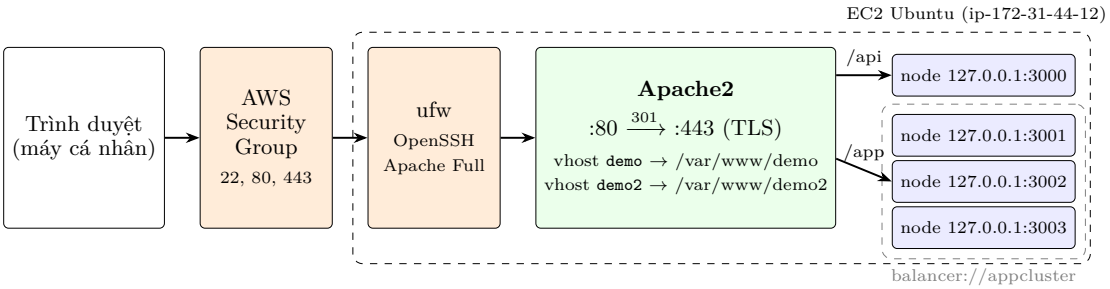
Bài thực hành được thực hiện trên một máy ảo **AWS EC2** (vùng `ap-southeast-1`, Singapore) chạy **Ubuntu**. Các thành phần chính:

Thành phần	**Sử dụng**
Web server	`apache2`, chạy dưới user `www-data`
Cấu hình virtual host	`/etc/apache2/sites-available/*.conf`, bật bằng `a2ensite`
Kiểm tra cấu hình	`apache2ctl configtest`, `apache2ctl -S`
Module	`ssl proxy proxy_http proxy_balancer lbmethod_byrequests headers` (bật bằng `a2enmod`)
Chứng chỉ TLS	`/etc/ssl/certs`, `/etc/ssl/private`
Backend	Node.js, chạy bằng systemd
Tường lửa	ufw (trong OS) và AWS Security Group (ngoài máy ảo)
Kiểm soát truy cập bắt buộc	AppArmor

Bảng 1: Môi trường triển khai

1.2 Sơ đồ hệ thống

Trên AWS, mỗi request từ trình duyệt phải đi qua **hai lớp tường lửa**: Security Group của AWS (ngoài máy ảo) rồi mới tới ufw (trong hệ điều hành). Điểm này là nguyên nhân của sự cố ở Mục 4.



Hình 1: Kiến trúc triển khai và hai lớp tường lửa trên AWS

2 Các phần thực hiện

2.1 Phần 0 – Chuẩn bị môi trường

Yêu cầu: Cho hai tên miền `demo.hcmus.edu.vn` và `demo2.hcmus.edu.vn` phân giải về máy chủ (lab không có DNS thật).

Thực hiện: Thêm vào `/etc/hosts` trên máy cá nhân (IP public của EC2), và trên chính EC2 trỏ hai tên miền về localhost:

```
# máy cá nhân
13.212.177.185  demo.hcmus.edu.vn demo2.hcmus.edu.vn
# trên EC2
127.0.0.1       demo.hcmus.edu.vn demo2.hcmus.edu.vn
```

2.2 Phần A – Cài đặt và baseline

Yêu cầu: Cài Apache, mở tường lửa, kiểm tra dịch vụ đang chạy.

Thực hiện:

```
apt update && apt install -y apache2
a2enmod ssl proxy proxy_http proxy_balancer lbmethod_byrequests headers
systemctl enable --now apache2
ufw allow OpenSSH && ufw allow 'Apache Full' && ufw enable
```

Ngoài ufw, trên AWS còn phải mở inbound cổng 80 và 443 trong Security Group của instance (Hình 4).

Kết quả:

```
root@ip-172-31-44-12:~# systemctl is-active apache2
active
root@ip-172-31-44-12:~# ss -tlnp | grep -E ":80|:443"
LISTEN 0      511                *:80              *:*    users:(("apache2",pid=6686,fd=4),("apache2",pid=6685,fd=4),("apache2",pid=6416,fd=4))
LISTEN 0      511                *:443             *:*    users:(("apache2",pid=6686,fd=6),("apache2",pid=6685,fd=6),("apache2",pid=6416,fd=6))
root@ip-172-31-44-12:~#
```

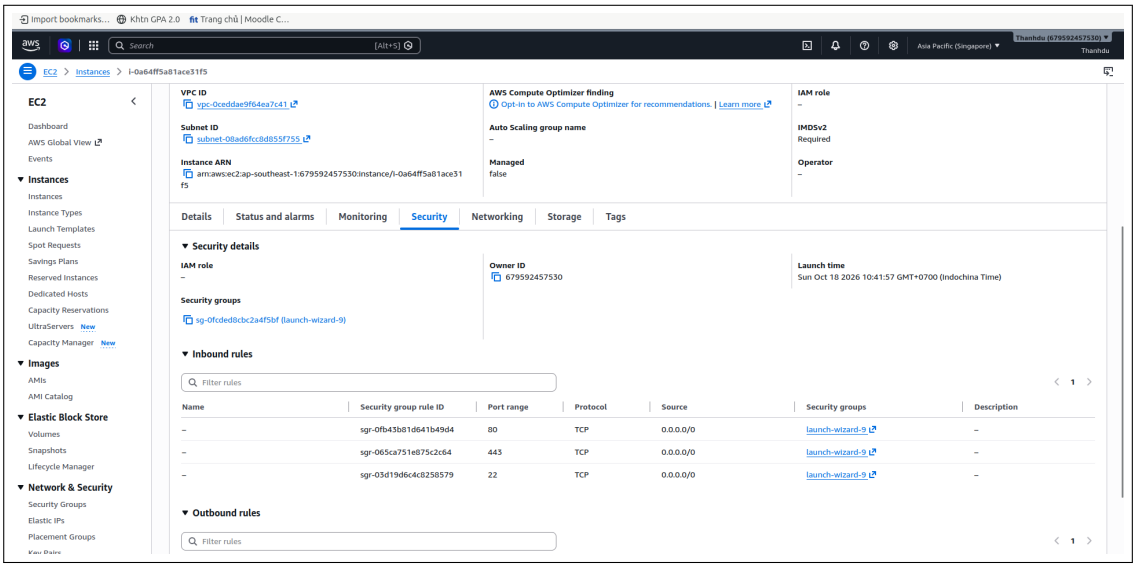
Hình 2: Apache đang chạy và lắng nghe cổng 80, 443

```
root@ip-172-31-44-12:~# ufw status
Status: active

To                         Action      From
--                         ------      ----
OpenSSH                    ALLOW       Anywhere
Apache Full                ALLOW       Anywhere
OpenSSH (v6)               ALLOW       Anywhere (v6)
Apache Full (v6)           ALLOW       Anywhere (v6)

root@ip-172-31-44-12:~#
```

Hình 3: ufw cho phép OpenSSH và Apache Full



Hình 4: Inbound rules của Security Group trên AWS: 22, 80, 443

2.3 Phần B – Website đầu tiên trên Virtual Host (demo)

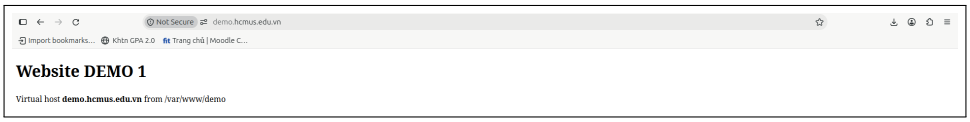
Yêu cầu: Tạo website tĩnh `/var/www/demo` phục vụ cho tên miền `demo.hcmus.edu.vn`.

Thực hiện:

```
mkdir -p /var/www/demo
# tạo /var/www/demo/index.html với tiêu đề "Website DEMO 1"
chown -R root:www-data /var/www/demo
chmod -R 750 /var/www/demo
a2ensite demo
apache2ctl configtest && systemctl reload apache2
```

Nội dung đầy đủ của `demo.conf` (sau khi đã thêm HTTPS, `/api`, `/app`) ở File 4.

Kết quả:



Hình 5: Vhost 1: `demo.hcmus.edu.vn`

2.4 Phần C – Virtual Host thứ hai (demo2)

Yêu cầu: Thêm website thứ hai trên cùng IP, Apache phân biệt hai site bằng header `Host:`.

Thực hiện: Tạo `/var/www/demo2` tương tự Phần B, tạo `demo2.conf` rồi `a2ensite demo2`. Site `demo2` cũng được bổ sung một VirtualHost `:443` với chứng chỉ riêng.

```
<VirtualHost *:80>
    ServerName demo2.hcmus.edu.vn
    DocumentRoot /var/www/demo2

    <Directory /var/www/demo2>
        Options -Indexes +FollowSymLinks
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/demo2_error.log
    CustomLog ${APACHE_LOG_DIR}/demo2_access.log combined
</VirtualHost>


<VirtualHost *:443>
    ServerName demo2.hcmus.edu.vn
    DocumentRoot /var/www/demo2

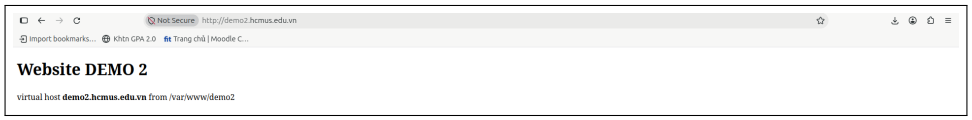
    SSLEngine on
    SSLCertificateFile    /etc/ssl/certs/demo2.crt
    SSLCertificateKeyFile /etc/ssl/private/demo2.key
    SSLProtocol -all +TLSv1.2 +TLSv1.3

    <Directory /var/www/demo2>
        Options -Indexes +FollowSymLinks
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/demo2_error.log
    CustomLog ${APACHE_LOG_DIR}/demo2_access.log combined
</VirtualHost>
```

File 1: `/etc/apache2/sites-available/demo2.conf`

Kết quả: Hai tên miền cho ra hai trang khác nhau dù cùng một IP và một tiến trình Apache. `apache2ctl -S` liệt kê hai namevhost trên cổng 80.



Hình 6: Vhost 2: `demo2.hcmus.edu.vn`

```
root@ip-172-31-44-12:~#  apache2ctl -S
VirtualHost configuration:
*:443                  demo.hcmus.edu.vn (/etc/apache2/sites-enabled/demo.conf:7)
*:80                   is a NameVirtualHost
         default server ip-172-31-44-12.ap-southeast-1.compute.internal (/etc/apache2/sites-enabled/000-default.conf:1)
         port 80 namevhost ip-172-31-44-12.ap-southeast-1.compute.internal (/etc/apache2/sites-enabled/000-default.conf:1)
         port 80 namevhost demo.hcmus.edu.vn (/etc/apache2/sites-enabled/demo.conf:1)
         port 80 namevhost demo2.hcmus.edu.vn (/etc/apache2/sites-enabled/demo2.conf:1)
ServerRoot: "/etc/apache2"
Main DocumentRoot: "/var/www/html"
Main ErrorLog: "/var/log/apache2/error.log"
Mutex watchdog-callback: using_defaults
Mutex proxy-balancer-shm: using_defaults
Mutex ssl-stapling-refresh: using_defaults
Mutex ssl-stapling: using_defaults
Mutex proxy: using_defaults
Mutex ssl-cache: using_defaults
Mutex default: dir="/run/apache2/" mechanism=default
PidFile: "/run/apache2/apache2.pid"
Define: DUMP_VHOSTS
Define: DUMP_RUN_CFG
User: name="www-data" id=33
Group: name="www-data" id=33
root@ip-172-31-44-12:~#
```

Hình 7: `apache2ctl -S`: hai namevhost trên cổng 80, demo trên 443

2.5 Phần D – Node API backend và Reverse Proxy (/api)

Yêu cầu: Dựng backend Node.js chỉ lắng nghe nội bộ, Apache làm reverse proxy cho đường dẫn `/api`.

Thực hiện: Backend `/opt/api/app.js` trả JSON cho biết instance nào xử lý request. Backend được chạy bằng systemd dạng instantiated service, mỗi cổng là một instance `nodeapi@<port>`.

```
const http = require('http');
const port = process.argv[2] || 3000;
http.createServer((req, res) => {
  res.setHeader('Content-Type', 'application/json');
  res.end(JSON.stringify({
    instance: `node-${port}`,
    path: req.url,
    pid: process.pid,
    time: new Date().toISOString()
  }));
}).listen(port, '127.0.0.1', () => console.log(`API on 127.0.0.1:${port}`));
```

File 2: `/opt/api/app.js`

```
[Unit]
Description=Node API on port %i
After=network.target

[Service]
ExecStart=/usr/bin/node /opt/api/app.js %i
Restart=always
User=www-data
WorkingDirectory=/opt/api

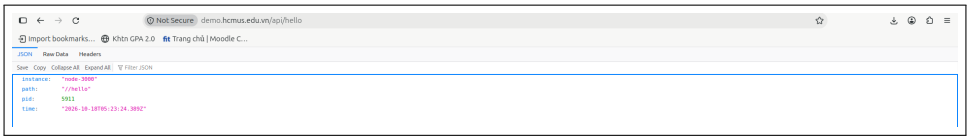
[Install]
WantedBy=multi-user.target
```

File 3: `/etc/systemd/system/nodeapi@.service`

```
systemctl daemon-reload
systemctl enable --now nodeapi@3000
```

Trong `demo.conf` thêm `ProxyPreserveHost`, `ProxyPass` và `ProxyPassReverse` cho `/api` (File 4).

Kết quả: `/api/hello` trả JSON từ `node-3000`, trong khi truy cập thẳng cổng 3000 từ bên ngoài thì không được: mọi request buộc phải đi qua Apache.



Hình 8: `/api/hello` trả JSON từ `node-3000`

```
thanhdu@EmDu:~$ curl -m 5  https://demo.hcmus.edu.vn:3000
curl: (28) Connection timed out after 5001 milliseconds
thanhdu@EmDu:~$
```

Hình 9: Từ máy ngoài không truy cập thẳng được cổng 3000

2.6 Phần E – Load Balancer (pool 3 backend + failover)

Yêu cầu: Chạy 3 instance backend, cấu hình Apache chia tải round-robin và tự né instance bị lỗi.

Thực hiện:

```
systemctl enable --now nodeapi@3001 nodeapi@3002 nodeapi@3003
```

Trong `demo.conf` khai báo `balancer://appcluster` gồm 3 thành viên với `lbmethod=byrequests` và ánh xạ đường dẫn `/app` (File 4). Thử failover bằng cách dừng `nodeapi@3002` rồi gửi lại request.

Kết quả: Response xoay vòng qua 3001, 3002, 3003; khi dừng 3002, traffic tự chuyển sang 3001 và 3003 mà client không thấy lỗi.

```
root@ip-172-31-44-12:~# ss -tlnp | grep -E ':300[0-3]'
LISTEN 0      511        127.0.0.1:3003      0.0.0.0:*    users:(("node",pid=7428,fd=21))
LISTEN 0      511        127.0.0.1:3002      0.0.0.0:*    users:(("node",pid=7425,fd=21))
LISTEN 0      511        127.0.0.1:3001      0.0.0.0:*    users:(("node",pid=7424,fd=21))
LISTEN 0      511        127.0.0.1:3000      0.0.0.0:*    users:(("node",pid=7423,fd=21))
root@ip-172-31-44-12:~#
```

Hình 10: Bốn instance Node chỉ lắng nghe trên 127.0.0.1

```
ubuntu@ip-172-31-44-12:~$ for i in $(seq 1 6); do curl -s http://demo.hcmus.edu.vn/app/ | grep -o 'node-[0-9]*'; done
node-3001
node-3002
node-3003
node-3001
node-3002
node-3003
ubuntu@ip-172-31-44-12:~$ systemctl stop nodeapi@3002
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ====
Authentication is required to stop 'nodeapi@3002.service'.
Authenticating as: Ubuntu (ubuntu)
Password:
[1]+  Stopped                 systemctl stop nodeapi@3002
ubuntu@ip-172-31-44-12:~$ sudo systemctl stop nodeapi@3002
ubuntu@ip-172-31-44-12:~$ for i in $(seq 1 6); do curl -s http://demo.hcmus.edu.vn/app/ | grep -o 'node-[0-9]*'; done
node-3001
node-3003
node-3003
node-3001
node-3003
node-3001
ubuntu@ip-172-31-44-12:~$ S
```

Hình 11: Load balancer xoay vòng 3 instance; khi dừng `nodeapi@3002` traffic tự chuyển sang 3001, 3003

2.7 Phần F – SSL/TLS (HTTPS) + redirect + HSTS

Yêu cầu: Bật HTTPS bằng chứng chỉ tự ký, ép redirect 80 sang 443 và bật HSTS.

Thực hiện:

```
openssl req -x509 -nodes -newkey rsa:2048 -days 365 \
  -keyout /etc/ssl/private/demo.key -out /etc/ssl/certs/demo.crt \
  -subj "/C=VN/ST=HCM/L=HCMC/O=HCMUS/CN=demo.hcmus.edu.vn"
```

VirtualHost `:80` chỉ còn nhiệm vụ redirect 301 sang HTTPS; VirtualHost `:443` phục vụ nội dung tĩnh, `/api`, `/app` và gửi header HSTS. Đây là nội dung cuối cùng của `demo.conf`:

```
<VirtualHost *:80>
    ServerName demo.hcmus.edu.vn
    Redirect permanent / https://demo.hcmus.edu.vn/
</VirtualHost>

# :443 là nơi phục vụ thật
<VirtualHost *:443>
    ServerName demo.hcmus.edu.vn
    DocumentRoot /var/www/demo

    SSLEngine on
    SSLCertificateFile    /etc/ssl/certs/demo.crt
    SSLCertificateKeyFile /etc/ssl/private/demo.key
    SSLProtocol -all +TLSv1.2 +TLSv1.3

    Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains"

    <Directory /var/www/demo>
        Options -Indexes +FollowSymLinks
        Require all granted
    </Directory>

    ProxyPreserveHost On
    ProxyPass        /api http://127.0.0.1:3000/
    ProxyPassReverse /api http://127.0.0.1:3000/

    <Proxy balancer://appcluster>
        BalancerMember http://127.0.0.1:3001
        BalancerMember http://127.0.0.1:3002
        BalancerMember http://127.0.0.1:3003
        ProxySet lbmethod=byrequests
    </Proxy>
    ProxyPass        /app balancer://appcluster/
    ProxyPassReverse /app balancer://appcluster/

    ErrorLog ${APACHE_LOG_DIR}/demo_error.log
    CustomLog ${APACHE_LOG_DIR}/demo_access.log combined
</VirtualHost>
```

File 4: `/etc/apache2/sites-available/demo.conf`

Kết quả: HTTP trả `301 Moved Permanently` về `https://`; HTTPS trả `200 OK` kèm `Strict-Transport-Security`.

```
root@ip-172-31-44-12:~#  curl -I http://demo.hcmus.edu.vn/
HTTP/1.1 301 Moved Permanently
Date: Sun, 18 Oct 2026 05:27:28 GMT
Server: Apache
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'
Location: https://demo.hcmus.edu.vn/
Content-Type: text/html; charset=iso-8859-1

root@ip-172-31-44-12:~# curl -kI https://demo.hcmus.edu.vn/
HTTP/1.1 200 OK
Date: Sun, 18 Oct 2026 05:27:34 GMT
Server: Apache
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'
Strict-Transport-Security: max-age=31536000; includeSubDomains
Last-Modified: Sun, 18 Oct 2026 03:55:33 GMT
ETag: "a7-65e155e8a7d8b"
Accept-Ranges: bytes
Content-Length: 167
Vary: Accept-Encoding
Content-Type: text/html

root@ip-172-31-44-12:~#
```

Hình 12: HTTP bị redirect 301 sang HTTPS; HTTPS trả 200 kèm `Strict-Transport-Security`

2.8 Phần G – Tường lửa và AppArmor: rà soát và làm cứng

Yêu cầu: Chỉ mở đúng cổng cần thiết, giữ cơ chế kiểm soát truy cập bắt buộc hoạt động, ẩn version Apache và thêm header bảo mật.

Thực hiện: Tường lửa chỉ mở 22, 80, 443 (Hình 3, 4). AppArmor giữ nguyên chế độ enforce. Header bảo mật đặt trong `hardening.conf` (bật bằng `a2enconf hardening`); `ServerTokens`, `ServerSignature`, `TraceEnable` đặt trong `security.conf`.

```
<IfModule mod_headers.c>
    Header always set X-Frame-Options "DENY"
    Header always set X-Content-Type-Options "nosniff"
    Header always set Content-Security-Policy "default-src 'self'"
</IfModule>
```

File 5: `/etc/apache2/conf-available/hardening.conf`

```
ServerTokens Prod
ServerSignature Off
TraceEnable Off
```

File 6: `/etc/apache2/conf-available/security.conf (các dòng có hiệu lực)`

Kết quả: Header `Server` chỉ còn `Apache` (đã ẩn version), ba header bảo mật xuất hiện.

```
root@ip-172-31-44-12:~#  aa-status | head -5
apparmor module is loaded.
182 profiles are loaded.
104 profiles are in enforce mode.
   /snap/snapd/26865/usr/lib/snapd/snap-confine
   /usr/bin/man
root@ip-172-31-44-12:~#  curl -kI https://demo.hcmus.edu.vn/ | grep -Ei 'server|x-frame|x-content|content-security'
  % Total    % Received % Xferd  Average Speed  Time    Time    Time   Current
                                 Dload  Upload  Total   Spent   Left   Speed
  0    167   0      0   0      0      0      0                              0
Server: Apache
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'
root@ip-172-31-44-12:~#
```

Hình 13: AppArmor đang hoạt động; header `Server` đã ẩn version và có 3 header bảo mật

3 Bảng kiểm tra tổng

#	Hạng mục	Lệnh kiểm chứng	Kết quả	Hình	
1	Apache chạy	`systemctl is-active apache2`	active, LISTEN :80/:443	2	**Đạt**
2	Firewall mở	`ufw status` + Security Group	Apache Full; SG 22/80/443	3, 4	**Đạt**
3	Vhost 1	trình duyệt `demo.hcmus.edu.vn`	trang DEMO 1	5	**Đạt**
4	Vhost 2	trình duyệt `demo2...`; `apache2ctl -S`	trang DEMO 2; 2 namevhost	6, 7	**Đạt**
5	Reverse proxy	`/api/hello`	JSON node-3000	8	**Đạt**
6	Load balancer	lặp `curl .../app/`	xoay vòng 3001/2/3	11	**Đạt**
7	Failover	dừng `nodeapi@3002`, lặp curl	chỉ còn 3001, 3003, không lỗi	11	**Đạt**
8	Redirect HTTPS	`curl -I http://demo...`	301 → https, có HSTS	12	**Đạt**
9	AppArmor	`aa-status`	module loaded, enforce	13	**Đạt**
10	Hardening	`curl -kI https://...`	`Server: Apache` + 3 header	13	**Đạt**

Bảng 2: Kết quả kiểm tra tổng

4 Sự cố gặp phải và cách xử lý

4.1 Không truy cập được HTTPS do Security Group thiếu cổng 443

Triệu chứng: Sau khi bật HTTPS (Phần F), mở `http://demo.hcmus.edu.vn` trên trình duyệt ở máy cá nhân thì bị chuyển sang `https://` rồi **treo cho đến khi hết thời gian chờ** (connection timed out), không hiện trang. Trong khi đó, trên chính EC2, `curl -kI https://demo.hcmus.edu.vn/` vẫn trả `HTTP/1.1 200 OK`.

Phân tích, khoanh vùng: Kiểm tra lần lượt từ trong ra ngoài theo đường đi của gói tin (Hình 1):

1. **Apache:** `apache2ctl configtest` báo `Syntax OK`; `ss -tlnp | grep :443` cho thấy `apache2` đang LISTEN trên `*:443`; `curl` nội bộ trả 200 ⇒ Apache và SSL hoạt động đúng.

2. **Tường lửa trong OS:** `ufw status` có rule `Apache Full` (= 80 + 443) ⇒ ufw không chặn.

3. **Nhật ký Apache:** `tail -f /var/log/apache2/demo_access.log` trong lúc tải lại trang: chỉ ghi nhận request tới cổng 80 (bị redirect 301), **không có** request HTTPS nào từ máy cá nhân tới được Apache; `demo_error.log` cũng không có lỗi SSL ⇒ gói tin bị chặn *trước khi* tới máy ảo.

4. **Kiểu lỗi:** *timeout* (gói tin bị bỏ đi âm thầm) chứ không phải *connection refused* (máy trả RST vì không có tiến trình nghe). Đây là dấu hiệu điển hình của một tường lửa ở giữa.

Nguyên nhân: Trên AWS, lưu lượng vào EC2 bị lọc bởi **hai lớp độc lập**: Security Group (ở hạ tầng AWS) và ufw (trong Ubuntu). Security Group lúc tạo instance chỉ có inbound cho cổng 22 và 80. Mở cổng 443 trên ufw là chưa đủ vì gói tin HTTPS đã bị Security Group loại bỏ trước khi đến được card mạng của máy ảo. Cổng 80 vẫn mở nên trình duyệt nhận được 301, nhưng bước tiếp theo sang 443 thì bị chặn, khiến lỗi trông như "trang HTTPS bị treo".

Cách sửa: Vào *EC2* → *Instances* → *Security* → *Security groups (launch-wizard-9)* → *Edit inbound rules*, thêm rule **HTTPS, TCP 443, nguồn 0.0.0.0/0**. Không cần khởi động lại Apache hay máy ảo; tải lại trang thì truy cập được ngay (trình duyệt cảnh báo chứng chỉ tự ký, đây là điều bình thường trong lab). Kết quả ở Hình 4.

Bài học:

- Trên cloud, "mở firewall" nghĩa là mở **cả hai lớp**: firewall trong OS và firewall của nhà cung cấp (Security Group).
- `curl` trên localhost chạy được không chứng minh được dịch vụ truy cập được từ ngoài; cần kiểm thử từ máy khác.
- Phân biệt *timeout* và *connection refused* giúp khoanh vùng nhanh: timeout → firewall chặn; refused → dịch vụ không lắng nghe.
- Access log không có dòng nào cho request là bằng chứng rõ ràng request không tới được Apache.

4.2 Các điểm phát hiện thêm khi rà soát cấu hình

Dư dấu / khi proxy: JSON trả về ở Hình 8 có `"path": "//hello"`. Nguyên nhân là `ProxyPass /api http://127.0.0.1:3000/`: vế trái không có `/` cuối còn vế phải có, nên `/api/hello` được ghép thành `http://127.0.0.1:3000//hello`. Cách sửa là để hai vế đồng nhất:

```
ProxyPass        /api/ http://127.0.0.1:3000/
ProxyPassReverse /api/ http://127.0.0.1:3000/
ProxyPass        /app/ balancer://appcluster/
ProxyPassReverse /app/ balancer://appcluster/
```

Lỗi chính tả trong unit systemd: Bản `nodeapi@.service` lúc làm lab ghi `WantedBy=mutil-user.target` thay vì `multi-user.target`. Các service vẫn chạy vì được khởi động thủ công bằng `systemctl enable --now`, nhưng target sai tên nên khi EC2 khởi động lại, backend sẽ không tự chạy và `/api`, `/app` trả 503. Cách sửa: đổi lại thành `multi-user.target` (bản nộp kèm ở File 3 đã sửa), chạy `systemctl daemon-reload` rồi `systemctl reenable nodeapi@300{0,1,2,3}`.

5 Kết luận

Đã triển khai thành công một web server Apache trên Ubuntu (AWS EC2) đáp ứng đủ 10 hạng mục kiểm tra: hai virtual host trên cùng một IP, reverse proxy tới backend Node.js chỉ lắng nghe nội bộ, load balancer ba instance có failover, HTTPS với redirect 301 và HSTS, tường lửa chỉ mở đúng cổng cần thiết và các header làm cứng. Bài học lớn nhất là trên môi trường cloud, tường lửa có nhiều lớp và cần kiểm thử từ bên ngoài chứ không chỉ từ localhost.